

Plataforma GitOps Kubernetes

Implementacao Codex

Prompt, arquitetura e guia de implementacao para uma plataforma GitOps Kubernetes entregue em fases: MVP funcional primeiro, depois observabilidade, seguranca, CI/CD, producao e backup.

kind

ArgoCD

Kustomize

NGINX Ingress

cert-manager

Headlamp

Grafana

Prometheus

Sealed Secrets

Velero

Documentos

01

DIAGRAMAS

Visao visual do ambiente, acessos, GitOps e monitoramento

02

PROMPT

Especificacao da implementacao Codex

03

ARQUITETURA

Camadas, paineis web e decisoes tecnicas

04

GUIA DE IMPLEMENTACAO

Fases incrementais e criterios de saida

VISAO VISUAL DO AMBIENTE

Diagramas

Componentes, acesso web, GitOps e observabilidade

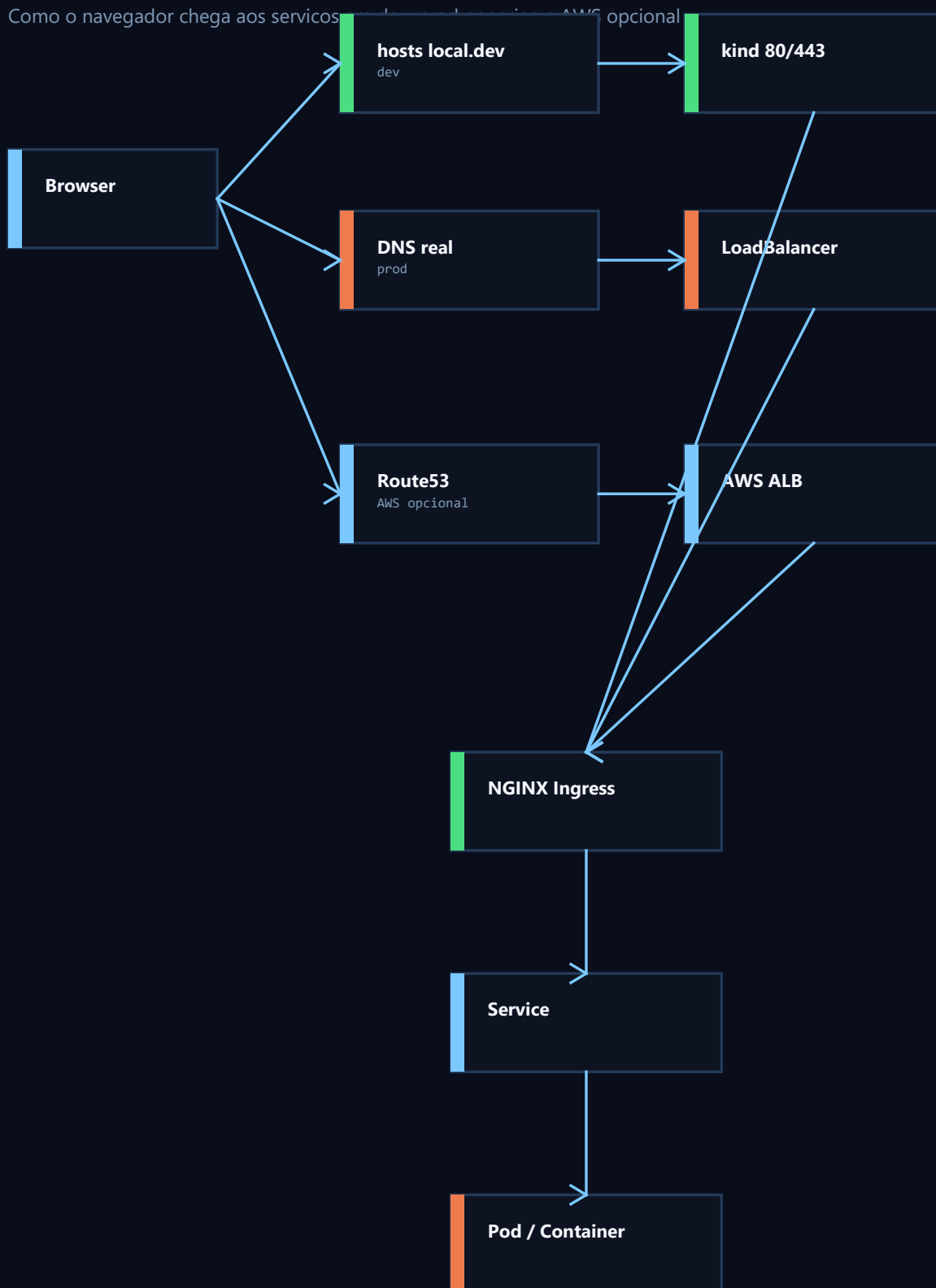
Visao de componentes

Repositorio GitOps reconciliado por ArgoCD nos ambientes dev e prod



Acesso web e entrada de trafego

Como o navegador chega aos serviços



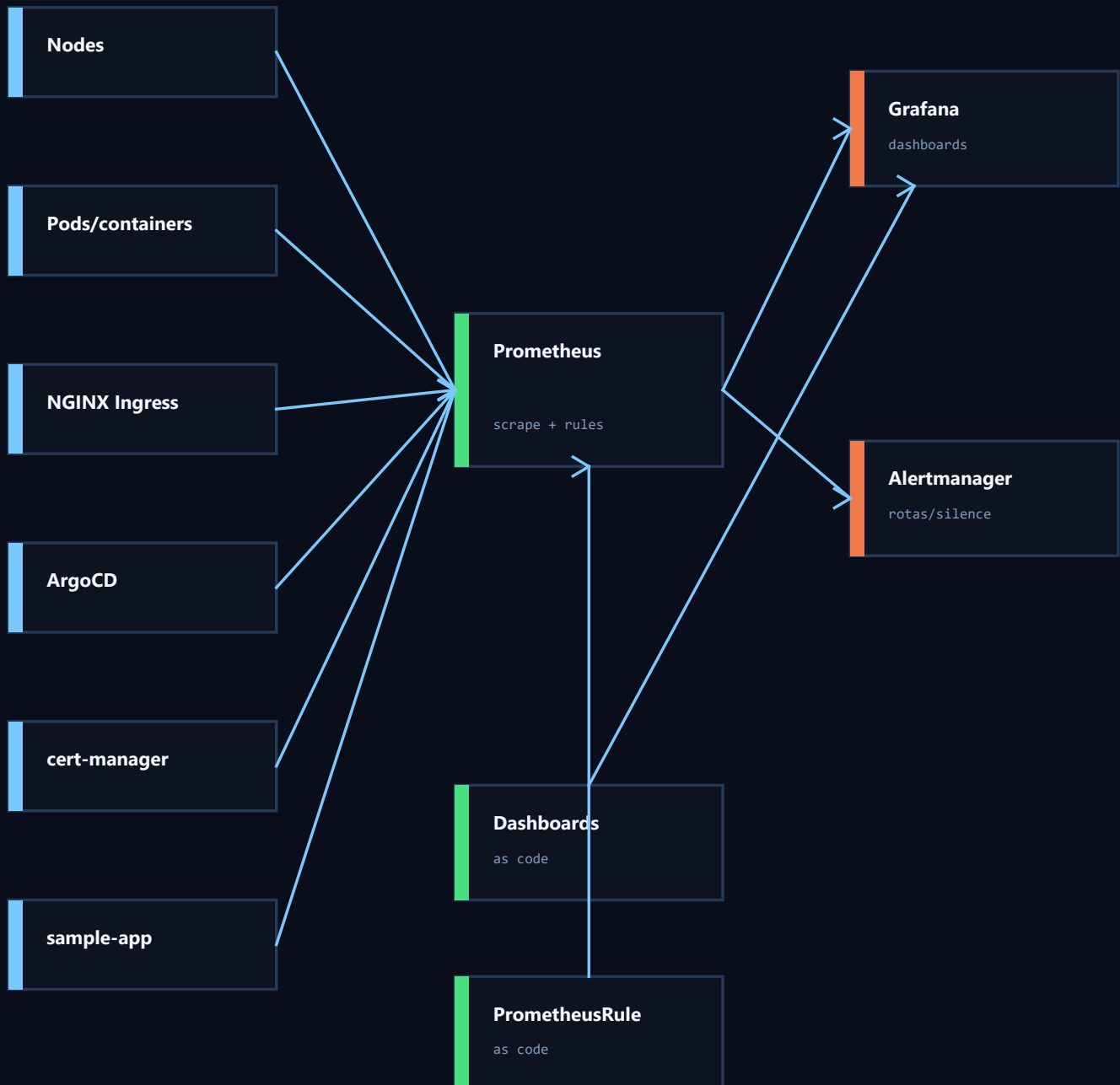
Fluxo GitOps e promocao

CI publica imagem e abre PR; ArgoCD sincroniza o Git no cluster



Observabilidade

Métricas e alertas versionados como código



DOCUMENTO 01 DE 03

PROMPT

Especificacao da implementacao Codex

PROMPT: Plataforma GitOps Kubernetes - Implementacao Codex

Objetivo

Voce e um Platform Engineer senior trabalhando neste repositorio

GitOps-LAB-Kubernetes-Observabilidade.

Sua tarefa e construir uma plataforma GitOps Kubernetes com dois objetivos:

- 1. Rodar localmente em ambiente de laboratorio usando kind.
- 2. Servir como base promovível para producao, com mudancas pequenas, auditaveis e revisadas.

O foco desta implementacao Codex e entregar primeiro um MVP GitOps funcional, e depois evoluir em camadas: observabilidade, seguranca, CI/CD, segredos, backup e producao.

Principio Central

O cluster deve refletir o Git.

Depois do bootstrap inicial do ArgoCD, nenhuma mudanca estrutural deve ser feita com kubectl apply manual. Alteracoes devem entrar no repositorio, passar por revisao e ser sincronizadas pelo ArgoCD.

Escopo MVP

O MVP deve entregar:

- Cluster local kind com 3 nos.
- ArgoCD instalado e acessivel via web.
- Padrao GitOps com infra/base e infra/overlays/dev.
- NGINX Ingress Controller.
- cert-manager com certificado self-signed para dev.
- Sample app publicada via HTTPS.
- Headlamp para visualizacao operacional do cluster.
- Grafana/Prometheus basicos para monitoramento.
- make up, make down, make status e make smoke-test.

Escopo Evolutivo

Apos o MVP, implementar:

- infra/overlays/prod renderizavel.
- NetworkPolicies default-deny com excecoes explicitas.
- Pod Security Standards: baseline em dev e restricted em prod.
- Sealed Secrets para o laboratorio.

GitHub Actions com build, test, scan Trivy, assinatura cosign e push GHCR.

- Promocao dev -> prod via Pull Request automatizado.
- Velero para backup e restore.
- Runbook operacional.

Stack

- Kubernetes local: kind.
- Kubernetes prod: qualquer cluster Kubernetes >= 1.28.
- GitOps: ArgoCD.
- Manifests: Kustomize base/ + overlays/<env>/.
- Ingress: NGINX Ingress Controller.
- TLS: cert-manager.
- Dashboard Kubernetes: Headlamp.
- Monitoramento: kube-prometheus-stack.
- Segredos: Sealed Secrets no MVP; External Secrets como evolucao.
- CI/CD: GitHub Actions, GHCR, Trivy, cosign.
- Backup: Velero.

Estrutura Esperada

```
.
??? clusters/
?   ??? dev/
?   ?   ??? kind-config.yaml
?   ?   ??? README.md
?   ??? prod/
?       ??? README.md
?
??? infra/
?   ??? base/
?   ?   ??? namespaces/
?   ?   ??? ingress-nginx/
?   ?   ??? cert-manager/
?   ?   ??? argocd/
```

Acessos Web no Ambiente Dev

Usar hosts explicitos, porque /etc/hosts nao suporta wildcard real.

```
https://argocd.local.dev
https://headlamp.local.dev
https://grafana.local.dev
https://prometheus.local.dev
https://alertmanager.local.dev
https://app.local.dev
```

Responsabilidade de Cada Painel

ArgoCD

Painel de GitOps.

Usado para:

- visualizar aplicacoes;
- comparar Git vs cluster;
- sincronizar aplicacoes;
- ver historico de deploy;
- diagnosticar erro de manifesto;
- acompanhar App of Apps e ApplicationSets.

ArgoCD ve pods e containers indiretamente, por meio de recursos Kubernetes como Deployment, ReplicaSet e Pod. Ele nao substitui um painel operacional de cluster.

Headlamp

Painel operacional Kubernetes.

Usado para:

- visualizar namespaces;
- visualizar pods;
- ver logs;
- inspecionar services, ingresses e configmaps;
- acompanhar events;
- entender o estado atual do cluster.

Grafana, Prometheus e Alertmanager

Painel de monitoramento e alerta.

Grafana e o painel principal. Prometheus e usado para PromQL, targets e regras.

Alertmanager gerencia alertas, rotas e silenciamentos.

Regras de Implementacao

- 1. Comecar pelo MVP.
- 2. Validar cada fase antes de seguir.
- 3. Evitar duplicacao entre base e overlays.
- 4. Nao guardar segredo puro no Git.
- 5. Nao usar cluster-admin para aplicacoes.
- 6. Todo workload deve ter resources.requests e resources.limits.
- 7. Producao deve ser cloud-agnostic por padrao.
- 8. Recursos especificos de AWS, Azure ou GCP devem ficar documentados como adaptacoes.

Critérios de Aceite do MVP

make up cria ou reutiliza o cluster dev.

- ArgoCD fica acessível em <https://argocd.local.dev>.
- Headlamp fica acessível em <https://headlamp.local.dev>.
- Grafana fica acessível em <https://grafana.local.dev>.
- Sample app responde em <https://app.local.dev>.
- ArgoCD mostra as aplicações Synced e Healthy.
- make smoke-test valida nodes, apps, certificados e endpoints.

Critérios de Aceite Final

- Dev sobe do zero de forma idempotente.
- Prod overlay renderiza sem erro.
- CI constrói, testa, escaneia, assina e publica imagem.
- Promoção para prod acontece por PR.
- Dashboards e alertas são versionados.
- NetworkPolicy default-deny ativa.
- PSS aplicado por ambiente.
- Sealed Secrets implementado.
- Velero configurado e restore documentado.
- docs/RUNBOOK.md cobre operações principais.

DOCUMENTO 02 DE 03

ARQUITETURA

Camadas, painéis web e decisões técnicas

Arquitetura - Plataforma GitOps Kubernetes Codex

Visao Geral

Esta arquitetura descreve uma plataforma GitOps em camadas.

A diferenca principal desta versao Codex e a ordem de entrega: primeiro um MVP pequeno, funcional e validavel; depois as capacidades avancadas.

O objetivo e evitar uma implementacao grande demais antes de provar o fluxo principal:

```
Git -> ArgoCD -> Kubernetes -> Ingress -> Aplicacao -> Monitoramento
```

Diagramas de Arquitetura

Visao de Componentes

Diagrama Mermaid versionado. A versao visual correspondente esta na secao Diagramas do PDF.

Acesso Web e Entrada de Trafego

Diagrama Mermaid versionado. A versao visual correspondente esta na secao Diagramas do PDF.

Fluxo GitOps e Promocao

Diagrama Mermaid versionado. A versao visual correspondente esta na secao Diagramas do PDF.

Observabilidade

Diagrama Mermaid versionado. A versao visual correspondente esta na secao Diagramas do PDF.

Camadas

1. Cluster

No ambiente dev, o cluster roda com kind.

Composicao:

- 1 control-plane.
- 2 workers.
- portas 80 e 443 mapeadas para o host.
- CNI padrao do kind no MVP.

Em producao, o repositorio nao cria o cluster. Ele apenas documenta os requisitos minimos:

Kubernetes >= 1.28.

- StorageClass dinamica.
- mecanismo de LoadBalancer.
- acesso a internet para imagens, Let's Encrypt e repositórios.
- permissão para instalar CRDs.

2. GitOps

ArgoCD é o reconciliador principal.

Existe uma fase inicial de bootstrap, na qual o script instala o ArgoCD. Depois disso, o ArgoCD assume o controle dos demais componentes.

Padrão:

- argocd/apps/app-of-apps-dev.yaml como raiz dev.
- argocd/apps/app-of-apps-prod.yaml como raiz prod.
- ApplicationSet para evolução multiambiente.

3. Manifests

Os manifests seguem o modelo:

```
infra/base
infra/overlays/dev
infra/overlays/prod
```

A base contém o que é comum.

O overlay contém apenas o que muda por ambiente.

Exemplos de diferenças:

Item	Dev	Prod
Domínio	*.local.dev com hosts explícitos	domínio real
TLS	self-signed	Let's Encrypt
Replicas	1	2 ou mais
Storage	local/pequeno	persistente
PSS	baseline	restricted
ArgoCD	simples	HA/SSO
Alertas	local	rotas reais

4. Entrada Web

No dev:

Browser

```
-> hosts locais
-> localhost:80/443
-> kind extraPortMappings
-> NGINX Ingress Controller
-> Service
```

Em producao cloud-agnostic:

Browser

```
-> DNS real
-> LoadBalancer Kubernetes
-> NGINX Ingress Controller
-> Service
-> Pod
```

Em AWS, isso pode ser adaptado para ALB usando AWS Load Balancer Controller, mas essa adaptacao fica documentada fora do caminho padrao para manter o projeto portavel.

Paineis Web

ArgoCD

URL dev:

```
https://argocd.local.dev
```

Funcao:

- GitOps;
- sync;
- diff Git vs cluster;
- historico de deploy;
- status de aplicacoes;
- arvore de recursos Kubernetes.

ArgoCD mostra pods e imagens, mas nao e um gerenciador de containers como Portainer.

Ele ve containers pela lente de Deployment, ReplicaSet e Pod.

Headlamp

URL dev:

```
https://headlamp.local.dev
```

Funcao:

- gestao visual do cluster;
- namespaces;
- pods;
- services;
- ingress;
- logs;
- events;
- troubleshooting operacional.

Grafana

URL dev:

```
https://grafana.local.dev
```

Funcao:

- dashboards;
- CPU/memoria por node e pod;
- reinicios;
- disponibilidade;
- metricas de ingress;
- metricas de ArgoCD;
- status de certificados.

Prometheus

URL dev:

```
https://prometheus.local.dev
```

Funcao:

- PromQL;
- targets;
- rules;
- metricas brutas.

Alertmanager

URL dev:

```
https://alertmanager.local.dev
```

Funcao:

- alertas ativos;
- silenciamentos;
- rotas de notificacao.

Seguranca

Seguranca entra em duas etapas.

No MVP:

- acesso local;
- TLS self-signed;
- credenciais iniciais documentadas;
- sem segredo puro em manifests.

Na fase de hardening:

- Sealed Secrets;
- NetworkPolicy default-deny;
- Pod Security Standards;
- RBAC minimo;
- assinatura de imagem;
- scan de vulnerabilidades;
- validacao futura com policy engine.

Observabilidade

O monitoramento usa kube-prometheus-stack.

Componentes:

- Prometheus;
- Grafana;
- Alertmanager;
- node-exporter;
- kube-state-metrics;
- ServiceMonitors.

Dashboards e alertas devem ser versionados no repositório.

CI/CD

O CI não aplica no cluster.

Fluxo esperado:

```
push
  -> GitHub Actions
  -> testes
  -> build
  -> Trivy scan
  -> cosign sign
  -> push GHCR
  -> PR alterando tag no overlay
  -> merge
  -> ArgoCD sync
```

Producao

O overlay prod deve ser renderizavel mesmo sem cluster de producao disponivel.

Ele deve preparar:

- TLS Let's Encrypt;
- replicas maiores;
- storage persistente;
- PSS restricted;
- configuracao para DNS real;
- Velero;
- ArgoCD com caminho para SSO/HA.

Decisoes Codex

- 1. MVP antes de plataforma completa.
- 2. Sealed Secrets como primeira solucao de segredos.
- 3. .github/workflows na raiz, nao dentro de pasta auxiliar.
- 4. Hosts explicitos no dev, porque /etc/hosts nao suporta wildcard.
- 5. Prod cloud-agnostic como padrao.
- 6. AWS ALB como adaptacao documentada, nao como dependencia inicial.

DOCUMENTO 03 DE 03

GUIA DE IMPLEMENTACAO

Fases incrementais e criterios de saida

Guia de Implementação - Plataforma GitOps Kubernetes Codex

Ordem Recomendada

Esta implementação deve ser feita em fases pequenas. Cada fase precisa ter um critério de saída objetivo antes de seguir para a próxima.

Fase 0 - Preparar o Repositorio

Criar:

- README.md
- Makefile
- clusters/dev/kind-config.yaml
- scripts/check-prerequisites.sh
- scripts/bootstrap-dev.sh
- scripts/teardown-dev.sh
- scripts/smoke-test.sh
- estrutura infra/base
- estrutura infra/overlays/dev
- estrutura argocd/apps

Critério de saída:

```
make status
```

deve rodar sem erro de sintaxe, mesmo que o cluster ainda não exista.

Fase 1 - Pre-requisitos

Validar:

- docker;
- kubectl;
- helm;
- kind;
- kustomize ou kubectl kustomize;
- jq;
- curl;
- git.

Ferramentas como cosign e trivy podem entrar na fase de CI/CD, para não bloquear o MVP.

Criterio de saida:

```
scripts/check-prerequisites.sh
```

mostra as versoes e falha com mensagens claras se algo estiver ausente.

Fase 2 - Cluster kind

Criar cluster dev:

```
kind create cluster --name gitops-dev --config clusters/dev/kind-config.yaml
```

O script deve ser idempotente:

- se o cluster nao existir, cria;
- se ja existir, reaproveita.

Criterio de saida:

```
kubectl get nodes
```

mostra 3 nos Ready.

Fase 3 - Bootstrap do ArgoCD

Instalar ArgoCD uma vez via script.

Depois de instalado, aplicar:

```
kubectl apply -f argocd/apps/app-of-apps-dev.yaml
```

A partir deste ponto, o restante deve ser gerenciado pelo ArgoCD.

Criterio de saida:

- namespace argocd existe;
- pods do ArgoCD estao Running;
- argocd-server responde por port-forward ou ingress temporario.

Fase 4 - App of Apps Dev

Criar as Applications iniciais:

- namespaces;
- ingress-nginx;

- cert-manager;
- sample-app.

No primeiro MVP, deixar monitoring, Headlamp e segurança avançada para fases seguintes.

Critério de saída:

```
kubectl get applications -n argocd
```

mostra apps Synced e Healthy.

Fase 5 - Ingress e TLS Dev

Instalar:

- NGINX Ingress Controller;
- cert-manager;
- ClusterIssuer self-signed;
- Certificates para os hosts locais.

Hosts dev:

```
argocd.local.dev  
headlamp.local.dev  
grafana.local.dev  
prometheus.local.dev  
alertmanager.local.dev  
app.local.dev
```

Criar dns/hosts-snippet.txt com entradas explícitas.

Critério de saída:

```
curl -k https://argocd.local.dev  
curl -k https://app.local.dev  
kubectl get certificate -A
```

Fase 6 - Sample App

Criar uma app simples com:

- endpoint /;
- endpoint /healthz;
- testes;
- Dockerfile;
- Deployment;
- Service;
- Ingress;

kustomization base/dev/prod.

Critério de saída:

```
curl -k https://app.local.dev/healthz
```

retorna sucesso.

Fase 7 - Headlamp

Instalar Headlamp via ArgoCD.

Dev:

- acesso por `https://headlamp.local.dev`;
- login por token de ServiceAccount.

Prod:

- documentar OIDC/SSO como caminho recomendado.

Critério de saída:

- página abre;
- token permite ver namespaces e pods.

Fase 8 - Observabilidade

Instalar kube-prometheus-stack.

Configurar:

- Grafana;
- Prometheus;
- Alertmanager;
- ServiceMonitor do ingress;
- ServiceMonitor do ArgoCD;
- ServiceMonitor da sample-app.

Criar dashboards iniciais:

- cluster overview;
- pods;
- ingress;
- ArgoCD;
- certificados.

Criar alertas iniciais:

- pod em CrashLoopBackOff;
- alto uso de memória;
- certificado expirando;

sync failed no ArgoCD;

- erro 5xx no ingress.

Criterio de saida:

- <https://grafana.local.dev> abre;
- dashboards aparecem sem import manual;
- Prometheus tem targets ativos.

Fase 9 - Smoke Test

Implementar scripts/smoke-test.sh validando:

- nodes Ready;
- ArgoCD Applications Synced/Healthy;
- certificates Ready;
- endpoints HTTPS;
- sample-app health;
- Grafana acessivel;
- Prometheus com targets.

Criterio de saida:

```
make smoke-test
```

termina com sucesso.

Fase 10 - Hardening

Adicionar:

- NetworkPolicy default-deny;
- allow DNS egress;
- allow ingress-nginx -> apps;
- allow Prometheus scrape;
- Pod Security labels;
- securityContext nos workloads;
- resources requests/limits.

Criterio de saida:

- pods continuam subindo;
- trafego nao permitido e bloqueado;
- workloads respeitam PSS.

Fase 11 - Segredos

Implementar Sealed Secrets.

Criar:

- controller;
- exemplo de secret selado;
- documentacao de criacao e rotacao.

Criterio de saida:

- nenhum secret puro no Git;
- secret e criado no cluster pelo controller.

Fase 12 - CI/CD

Criar .github/workflows/ci.yaml.

Pipeline:

- checkout;
- teste;
- build;
- Trivy scan;
- cosign sign keyless;
- push GHCR.

Criar .github/workflows/cd-promote.yaml.

Promocao:

- workflow abre PR alterando images.newTag;
- ArgoCD sincroniza apos merge.

Criterio de saida:

- imagem publicada no GHCR;
- PR de promocao criado;
- overlay atualizado por PR.

Fase 13 - Producao

Criar infra/overlays/prod.

Prod deve configurar:

- replicas maiores;
- TLS Let's Encrypt;
- ingress com LoadBalancer;
- storage persistente;
- PSS restricted;
- valores de recursos mais realistas;
- external-dns documentado;
- OIDC documentado.

Criterio de saida:


```
kubect1 kustomize infra/overlays/prod
```

renderiza sem erro.

Fase 14 - Backup e Runbook

Instalar Velero.

Dev:

- documentar teste com MinIO ou modo simulado.

Prod:

- S3-compatible object storage.

Criar docs/RUNBOOK.md com:

- restaurar backup;
- rotacionar secret;
- recuperar ArgoCD;
- diagnosticar certificado;
- diagnosticar app fora de sync;
- diagnosticar pod sem rede.

Criterio de saída:

- restore documentado;
- procedimento testado pelo menos uma vez quando houver backend de storage.

Resultado Esperado

Ao final:

```
make up
make smoke-test
```

devem entregar uma plataforma local funcional, com acesso web para:

- ArgoCD;
- Headlamp;
- Grafana;
- Prometheus;
- Alertmanager;
- sample-app.

E o overlay prod deve estar pronto para adaptacao em um cluster real.